

DevOps and Microservices

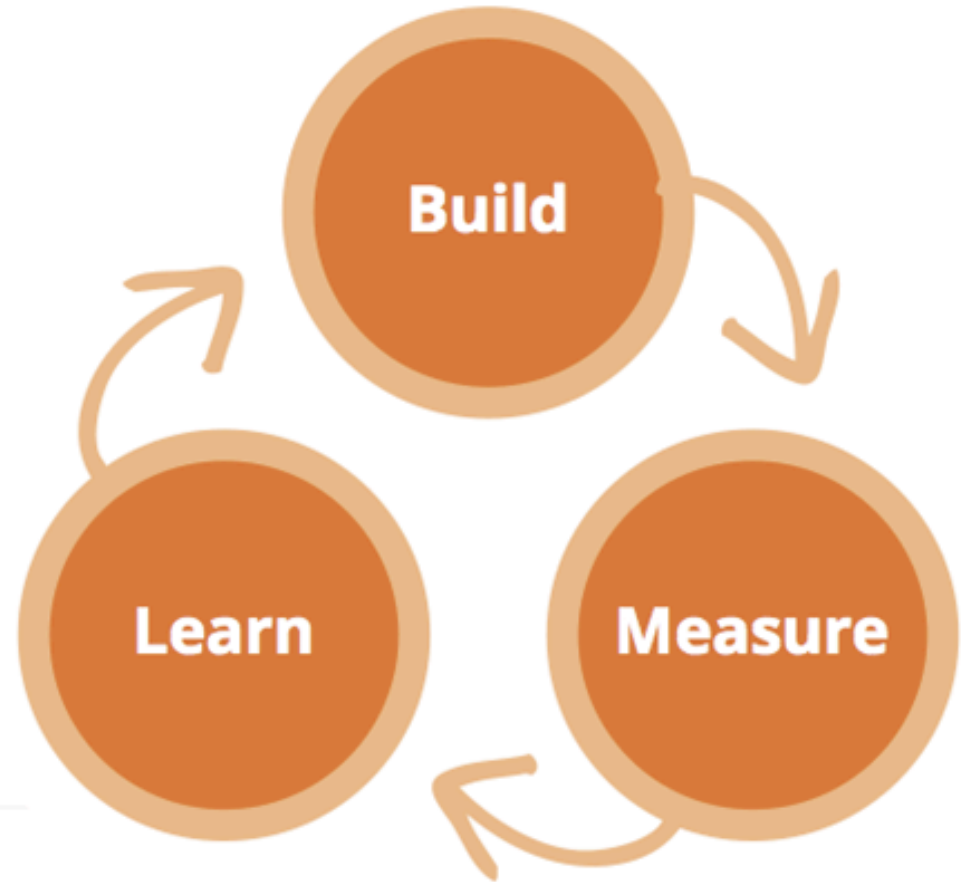
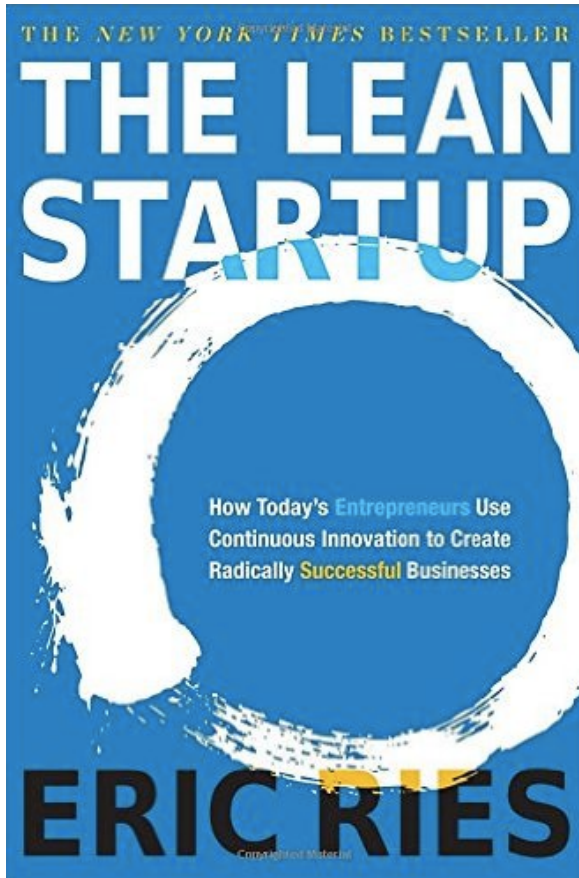
Cristian Klein



Agenda

- Motivation
- DevOps: Definition, Concepts
- Star as Code
- DevOps: Caveats
- Microservices

Why DevOps from 50,000 feet



Google Search Example

- **Business Hypothesis:**
 - “Users want 30 results instead of 10”
- **Build (A/B testing):**
 - Users A served 30 results
 - Users B served 10 results
 - Measure amount spent on Google Search
- **Measure:**
 - Users A spent **less time** on Google Search.
- **Learn:**
 - Displaying 30 results takes longer than 10 results
 - Display only 10 results OR make 30 results faster

Some Ideas of Lean Startup

- Faster **learning** => more **competitive**
- **Discover** requirements
 - Instead of debating them

OPERATIONS

GETS PAGED TO
FIX LIGHTBULB
AT 2 A.M.



HANDS OUT
FLASHLIGHTS



GOES BACK
TO SLEEP



BACKEND DEVELOPER

WONDERS WHY
EVERYBODY'S HAVING
PROBLEMS WITH
LIGHTBULBS



REALIZES HE FORGOT
TO DEPLOY LIGHT TO
DATABASE



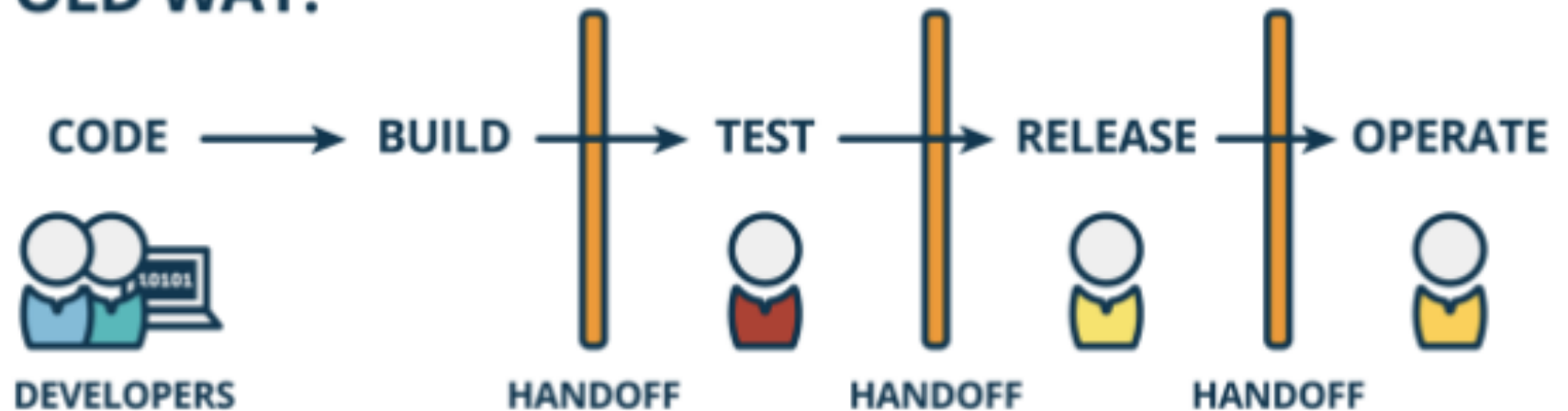
KEEP CALM &
BLAME FRONTEND



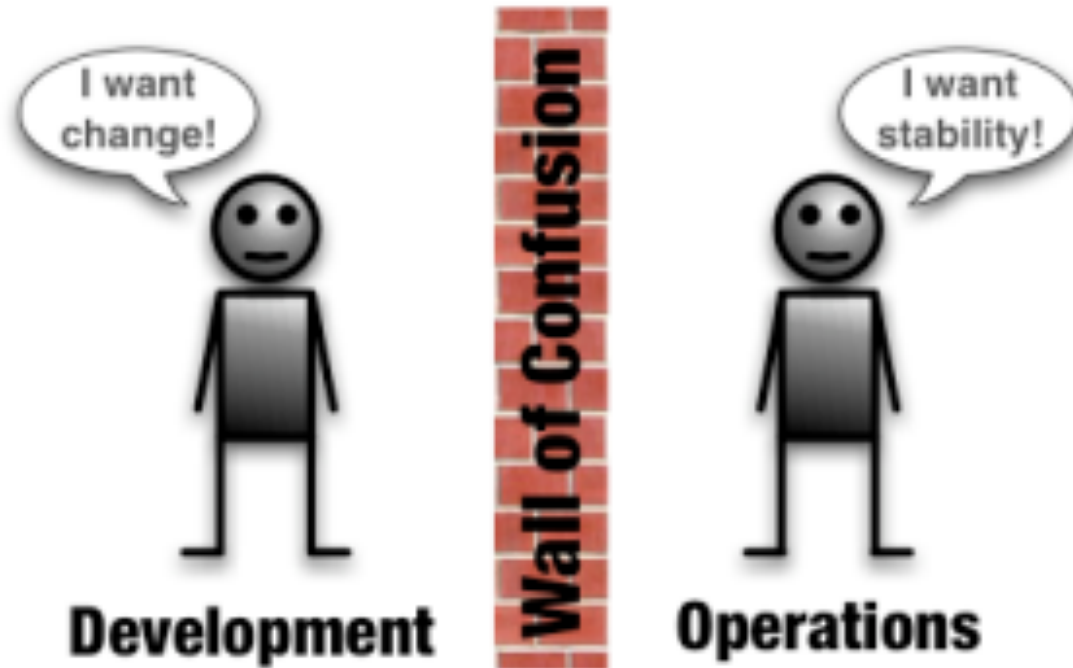
MART VIRKUS '16

TOGGL.COM

OLD WAY:



Misalignment of Incentives



Wall of Confusion



Sure glad the hole isn't at our end.



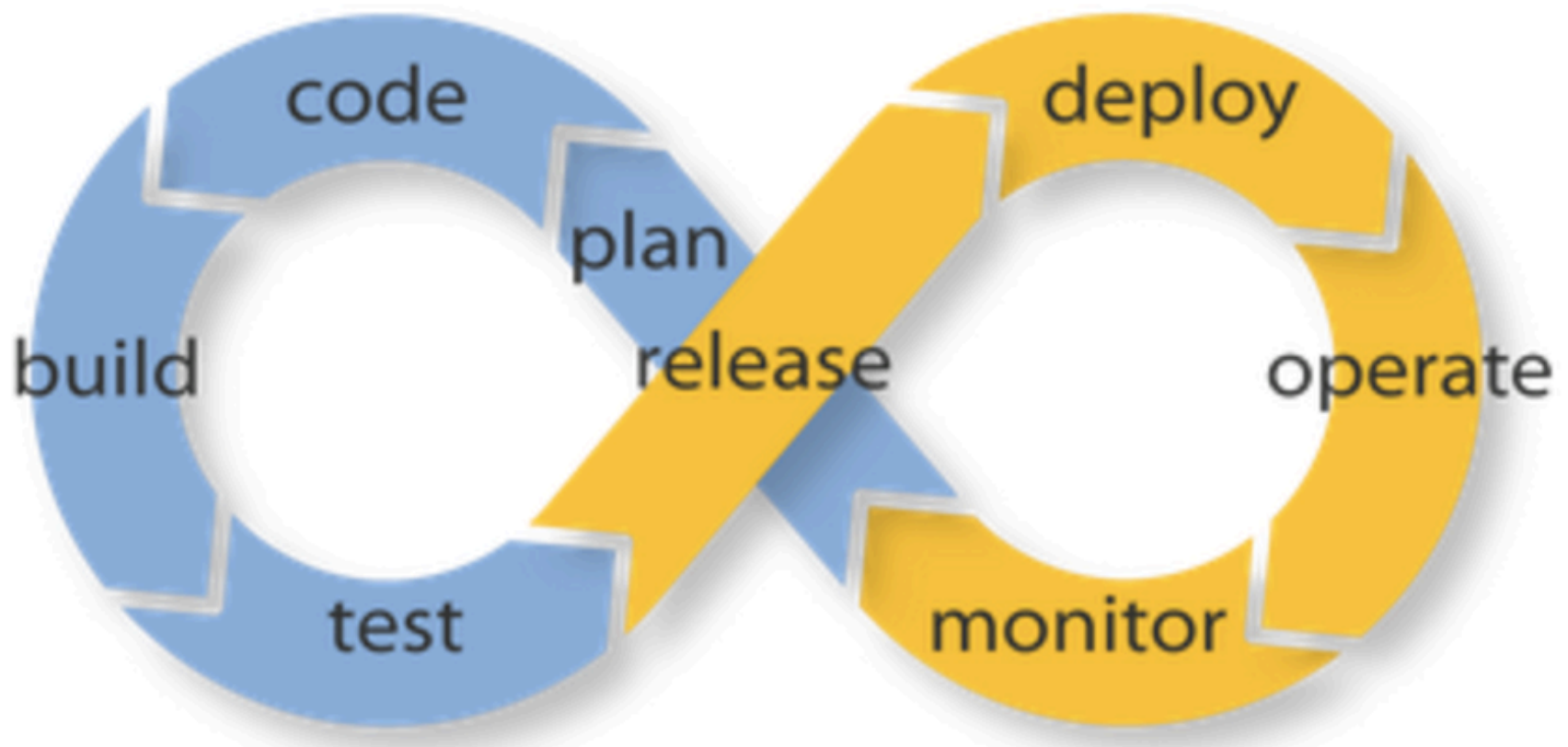
Agenda

- Motivation
- **DevOps: Definition, Concepts**
- Star as Code
- DevOps: Caveats
- Microservices

DevOps: Definition

- "DevOps" is an emerging set of *principles, methods and practices* for **communication, collaboration** and **integration** between **software development** (application/software engineering) and **IT operations** (systems administration/infrastructure) professionals.

DevOps “Pipeline”



DevOps (cont.)

- Acknowledges the **interdependence** of software development and IT operations
- Aims to help organizations **rapidly produce quality** software products and services
- Responds to the **demands of stakeholders** for an increased rate of production releases
- Supports the use of **agile development** processes
- **Aligns** Dev and Ops incentives

Downtime Budget Example

- Google's Site Reliability Engineers:
 - “SRE is what happens when you ask a software engineer to **design an operations team.**”
- “A is allowed to be down 15 minutes in a month.”
- Downtime budget spend => no more new features
- Devs are incentivized to:
 - invest more in **testing, resilience, automation, ...**
- Ops:
 - get paged **less often** ...
 - ... only for truly **new issues**

State of DevOps Report

- Annual report by Puppet
- Survey from 2016
- Companies that uses DevOps vs they who don't.



200x more frequent
deployments



24x faster
recovery from failures



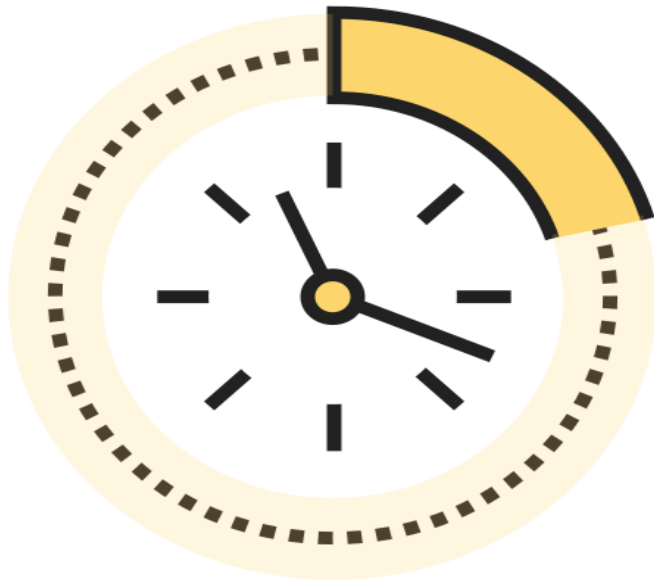
3x lower
change failure rate



2,555x shorter lead
times

Lead time - time between commit and production

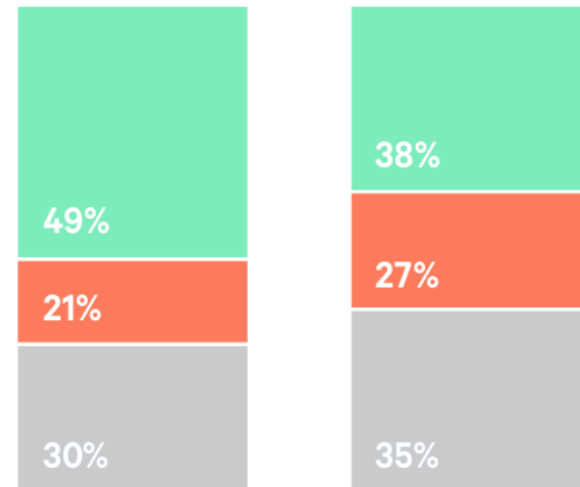
High performers spend



22%

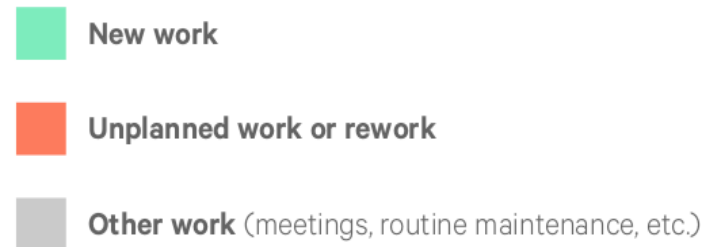
less time on
unplanned work and rework

New work vs. unplanned work



High performers

Low performers



Apart from that?

Worker **2.2** times more likely to recommend their organization to a friend as a good workplace



Agenda

- Motivation
- DevOps: Definition, Concepts
- **Star as Code**
- Micro-services

Star as Code

- **Pipeline** as Code
- **Infrastructure** as Code
- **Configuration** as Code

Lessons Learned from 597 outages 2009-2015

§	Root cause	#Sv	Cnt	%	Cnt '09-'15
	UNKNOWN	29	355	-	M.M.M.M.M.M.M
5.1	UPGRADE	18	54	16	7.4.M.5.M.4.7
5.2	NETWORK	21	52	15	4.4.6.8.M.8.5
5.3	BUGS	18	51	15	M.4.9.8.9.9.2
5.4	CONFIG	19	34	10	2.2.7.2.5.M.4
5.5	LOAD	18	31	9	2.5.5.5.4.8.2
5.6	CROSS	14	28	8	-.2.4.M.5.3.4
5.7	POWER	11	21	6	5.4.3.5.3.1.-
5.8	SECURITY	9	17	5	7.-.2.1.3.4.-
5.9	HUMAN	11	14	4	-.1.4.4.2.1.2
5.10	STORAGE	4	13	4	2.-.-.3.5.3.-
5.11	SERVER	6	11	3	-.3.-.2.2.4.-
5.12	NATDIS	5	9	3	1.1.3.2.1.1.-
5.11	HARDWARE	4	5	1	1.-.-.3.1.-.-

Motivation

Many DevOps tasks are:

- Dull
- Error-prone
- Stressful
- Time-consuming

Such as:

- Server **crafting**
- Software **configuration**
- **Deploying** code
- Solution: **Automate!**

Why “as code”?

- Code is:
 - **human** readable
 - **machine** executable
- Can be:
 - **versioned**
 - **audited**
 - **commented**
 - **tested**
 - **scaled** (configure 10,000 servers)

Playbooks instead of Tribal Knowledge



Approaches to “as code”

- Imperative (how?)
 - **Copy** this file
 - **Install** this package
- Declarative (what?)
 - **Ensure** this file is up-to-date
 - **Ensure** this package is installed and up-to-date
- Most tools offer both

Pipeline as Code: Jenkins



```
stage('Checkout'){  
    checkout scm  
}
```

```
stage('Test'){  
    env.NODE_ENV = "test"  
  
    sh 'node -v'  
    sh 'npm prune'  
    sh 'npm install'  
    sh 'npm test'  
}
```

```
stage('Build Docker'){  
    sh './dockerBuild.sh'  
}
```

```
stage('Deploy'){  
    echo 'Push to Repo'  
    sh './dockerPushToRepo.sh'
```

```
    sh 'ssh deploy@xxxxxx.com dockerRun.sh'  
}
```

```
stage('Cleanup'){  
    sh 'npm prune'  
    sh 'rm node_modules -rf'
```

```
    mail body: 'project build successful',  
          from: 'xxxx@yyyyyy.com',  
          replyTo: 'xxxx@yyyyyy.com',  
          subject: 'build successful'  
          to: 'yyyyyy@yyyyyy.com'
```

```
}
```

Pipeline as Code: Travis



`sudo: required`

`language: ruby`

`services:`

- `- docker`

`before_install:`

- `- docker pull carlad/sinatra`
- `- docker run -d -p 127.0.0.1:80:4567`
`carlad/sinatra /bin/sh -c "cd /root/sinatra; bundle exec foreman start;"`
- `- docker ps -a`
- `- docker run carlad/sinatra`
`/bin/sh -c "cd /root/sinatra; bundle exec rake test"`

`script:`

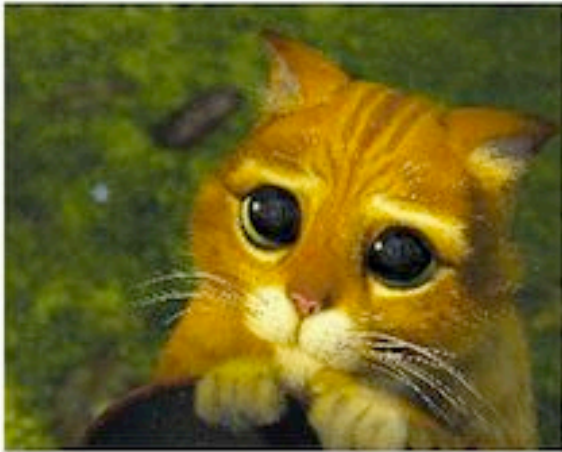
- `- bundle exec rake test`

Configuration Management as Code

- Describe systems **once**
- **Predictable** results
- Handle **differences** in environments
 - Dev vs. staging vs. production
- No **Snowflakes!**
- Server configuration time
 - Seconds



Infrastructure as Code: Pets vs. Cattle



- Pets are given names like `pussinboots.cern.ch`
- They are unique, lovingly hand raised and cared for
- When they get ill, you nurse them back to health



- Cattle are given numbers like `vm0042.cern.ch`
- They are almost identical to other cattle
- When they get ill, you get another one

IaC and CMaC Tools



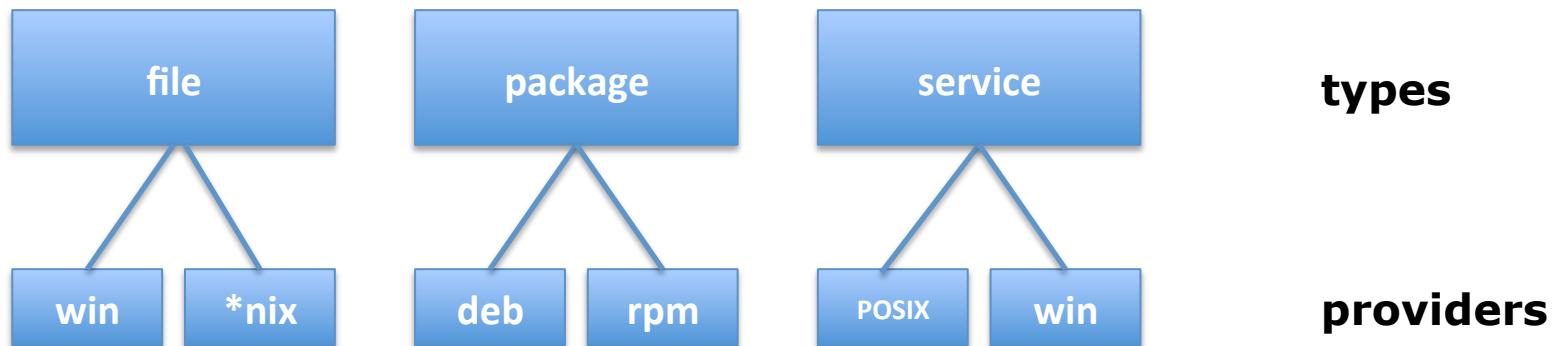
ANSIBLE



- Declarative configuration language
- Plain-text configuration in source control
- Fully programmatic, no manual interactions

Puppet overview

1. Describe what you want to be configured
2. (Don't care how it is done)
3. Describe dependencies



Puppet DSL

Puppet example: Apache server

- Manifests
- Resources
 - Name
 - Attributes and values
- Ordering and dependencies

```
package { "httpd":  
  name      => "httpd.x86_64",  
  ensure    => "present",  
}  
  
file { "http.conf":  
  path      => "/etc/httpd/conf/httpd.conf",  
  owner     => root,  
  group     => root,  
  mode      => 0644,  
  source    => "puppet:///modules/apache/httpd.conf",  
  require   => Package["httpd"],  
}  
  
service { "httpd":  
  ensure    => running,  
  enable    => true,  
  subscribe => File["http.conf"],  
}
```

Puppet building blocks

- Manifests
- Variables and (custom) facts
- Node declarations
- Classes and Modules
- Defined resource types
- Templates
- Console

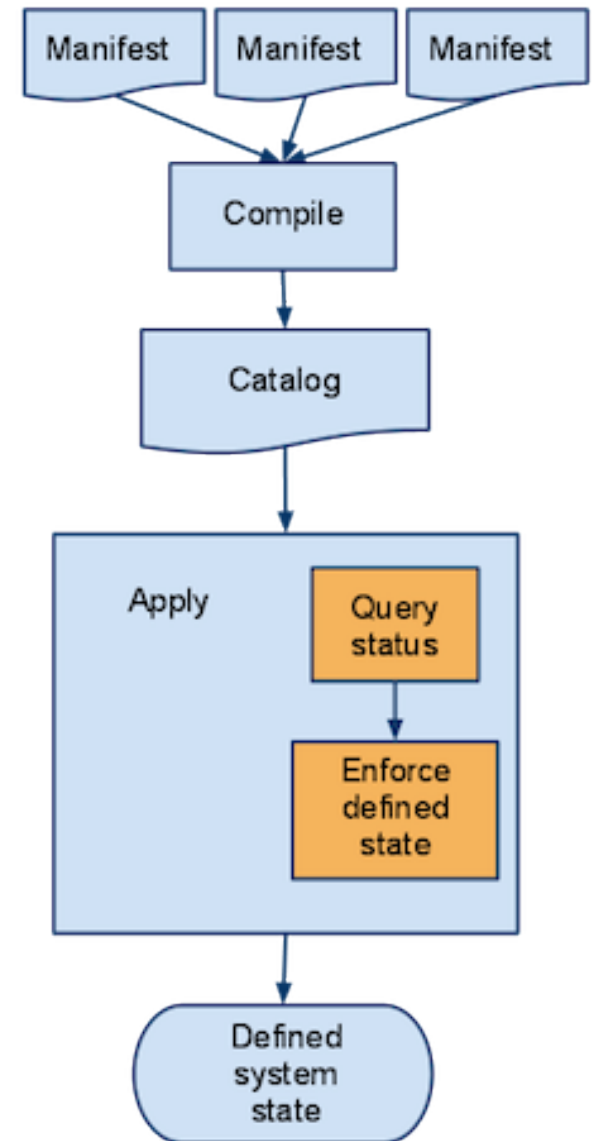
```
if $operatingsystem == 'CentOS'
```

```
node 'www1.example.com' {  
    include common  
    include apache  
}  
node 'db1.example.com' {  
    include common  
    include mysql  
}
```

```
file { "http.conf":  
    path      => "/etc/httpd/conf/httpd.conf",  
    owner     => 'root',  
    group     => 'root',  
    mode      => '0644',  
    content   => template('config/httpd.erb'),  
}
```

A Puppet run

1. Retrieve plugins from server
2. Get “facts” on client and send them to master
3. Compile catalog and send it to the client
4. Apply catalog on client
5. Process report



Challenge: Idempotence

Applying the same playbook twice **MUST** give the same results

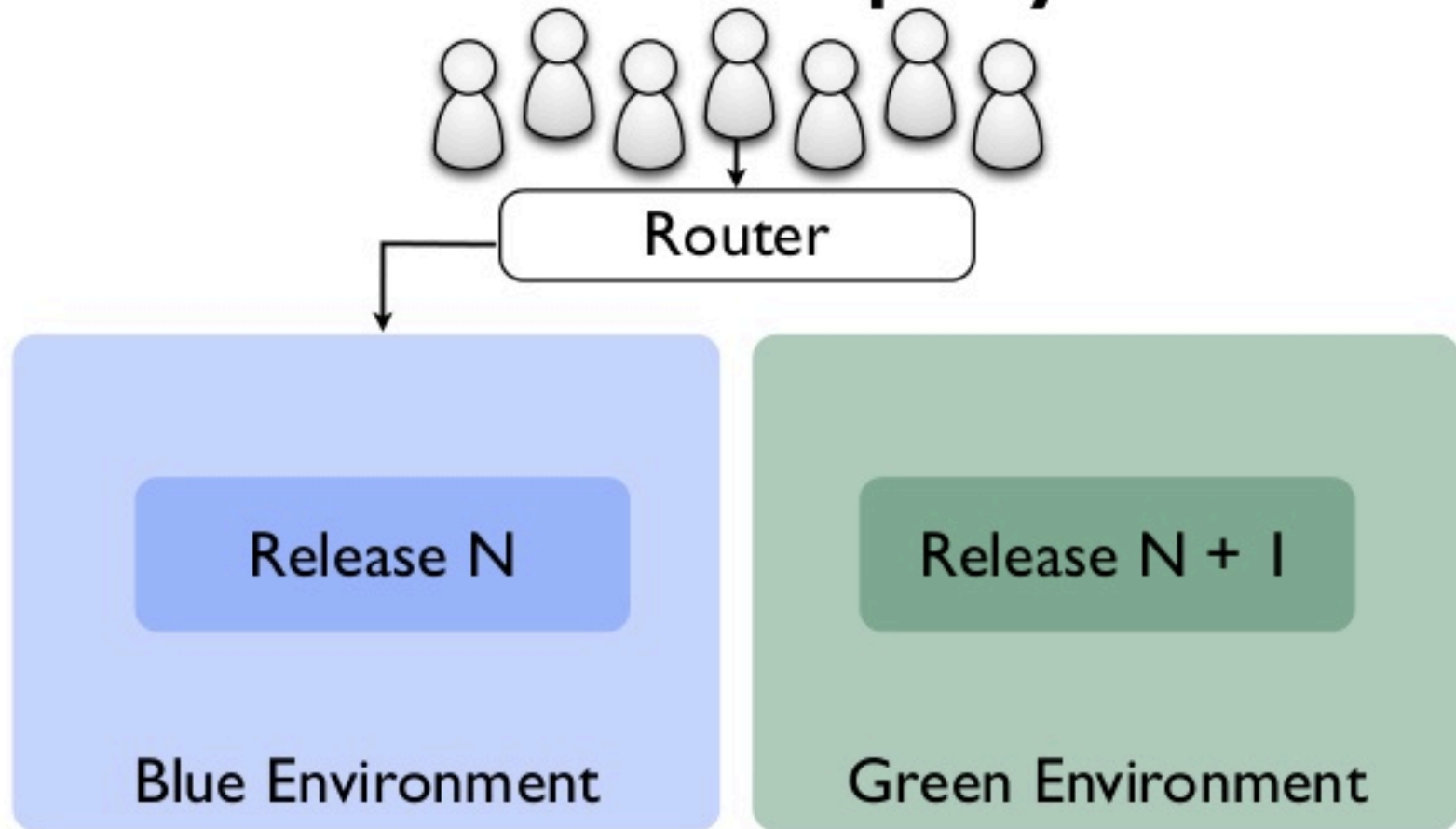
Tested Cookbooks	298
Number of Test Cases	3671
Number of Tasks (min/max/total)	1 / 103 / 4112
Total Task Executions	187986
Captured State Changes	164117
Total Non-Idempotent Tasks	263
Cookbooks With Non-Idempotent Tasks	92
Overall Net Execution Time	25.7 CPU-days
Overall Gross Execution Time	44.07 CPU-days

Hummer, Waldemar, et al. "Testing idempotence for infrastructure as code." Distributed Systems Platforms and Open Distributed Processing. 2013.

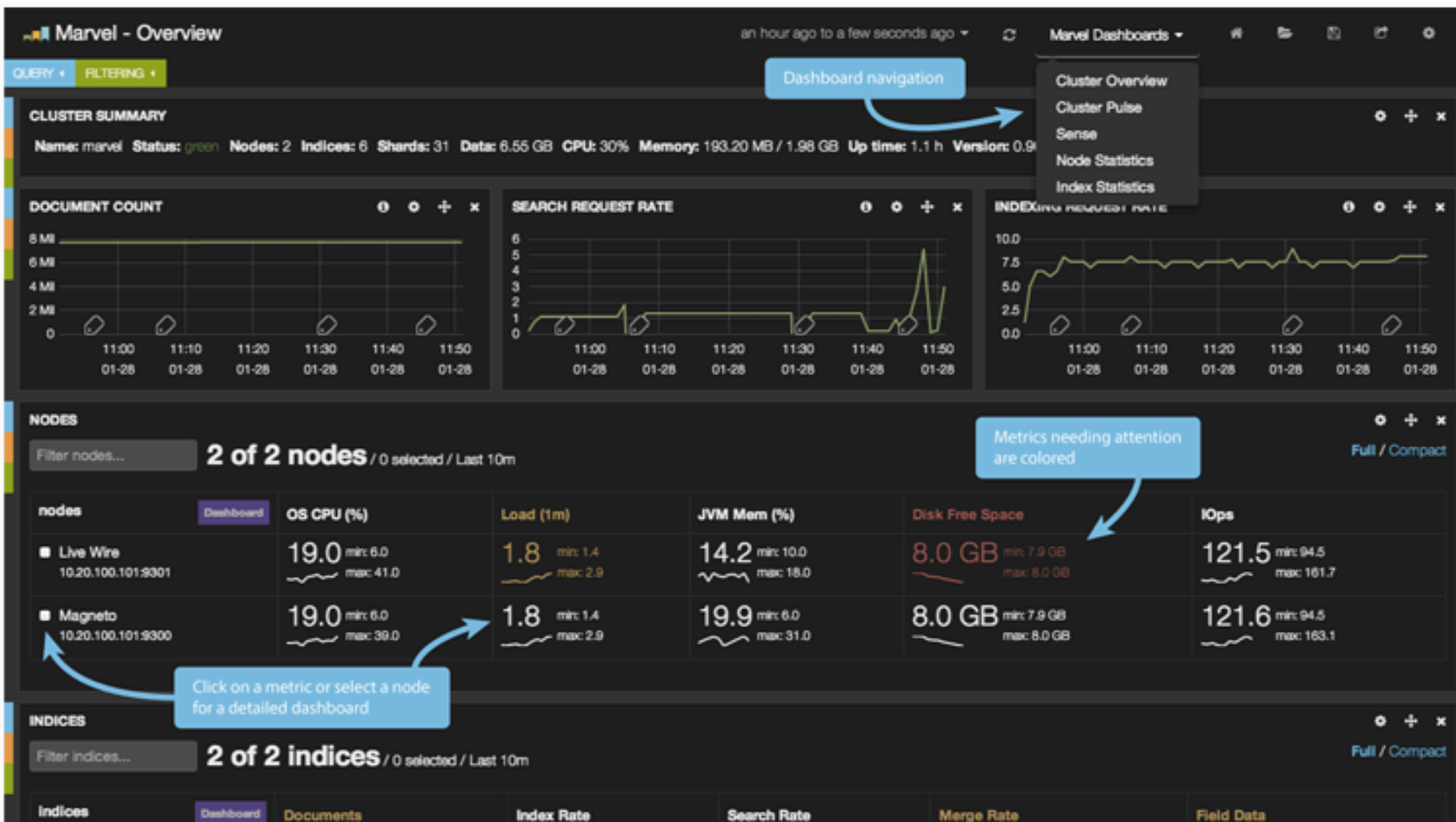
DevOps: Dangers

- “To err is human, but to err and deploy it to all systems simultaneously is DevOps”
 - A. Cockcroft, prev. Netflix Architect
- 2014 Gmail outage:
 - “The incorrect configuration was sent to live services over the next 15 minutes, caused users’ requests for their data to be ignored, and those services, in turn, generated errors.”
- 2016 Amazon WS S3 outage:
 - “Unfortunately, one of the inputs to the command was entered incorrectly and a larger set of servers was removed than intended.”
- Don’t roll out system-wide until it works...

Blue Green Deployments



Continuous Monitoring



Agenda

- Motivation
- DevOps: Definition, Concepts
- Star as Code
- **Microservices**

Microservices

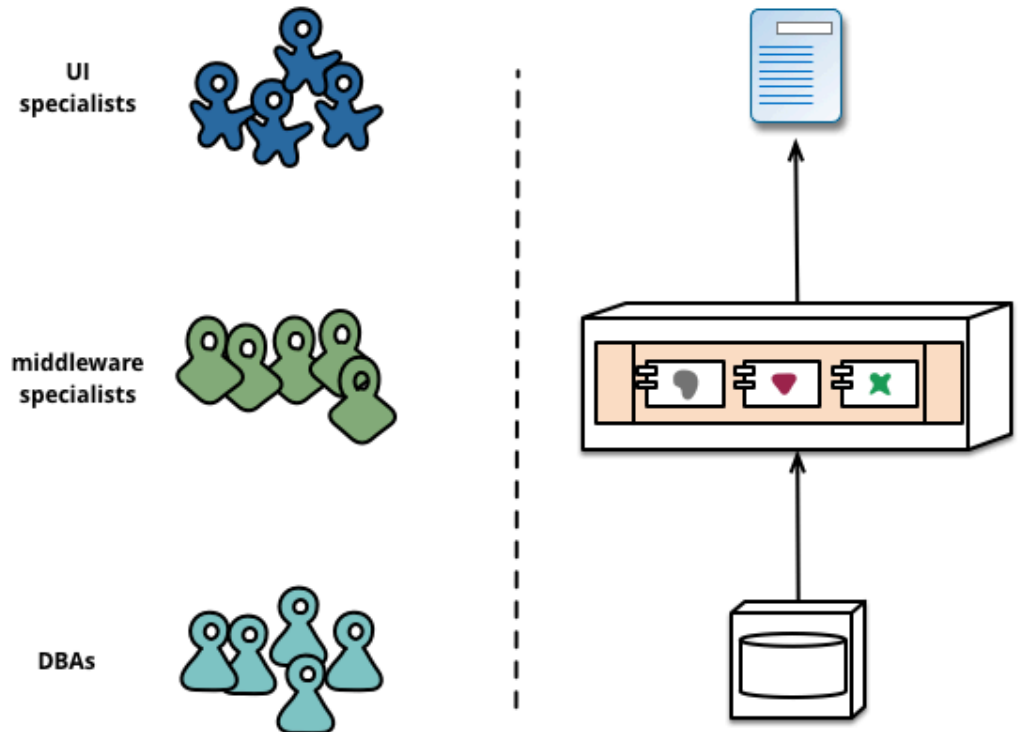
- (One) Definition: “Loosely coupled service oriented architectures with bounded contexts”
 - Loosely coupled: asynchronous service updates
 - Bounded context: Limited knowledge of surrounding services

Microservices: More about coupling

- **Organizational** coupling
- **Centralized** database schemes
- Enterprise service bus – **centralized** messaging

Microservices: Conway's Law

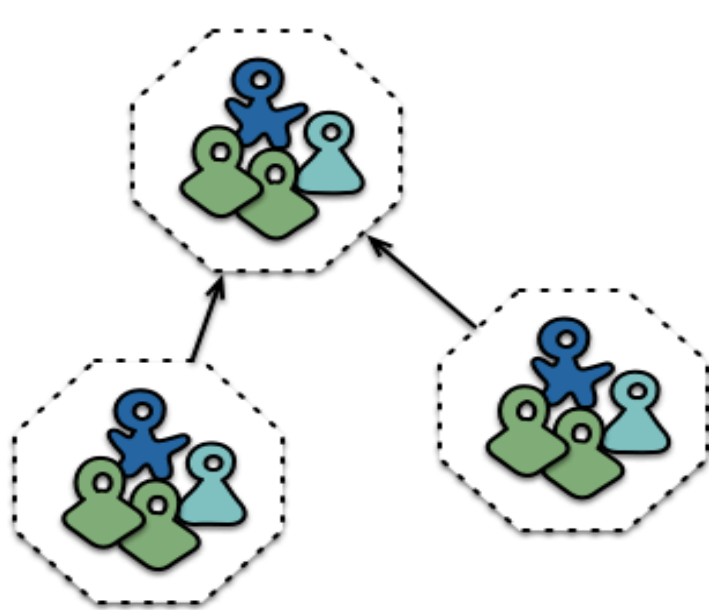
- Conway's law (1967):
 - “Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure”



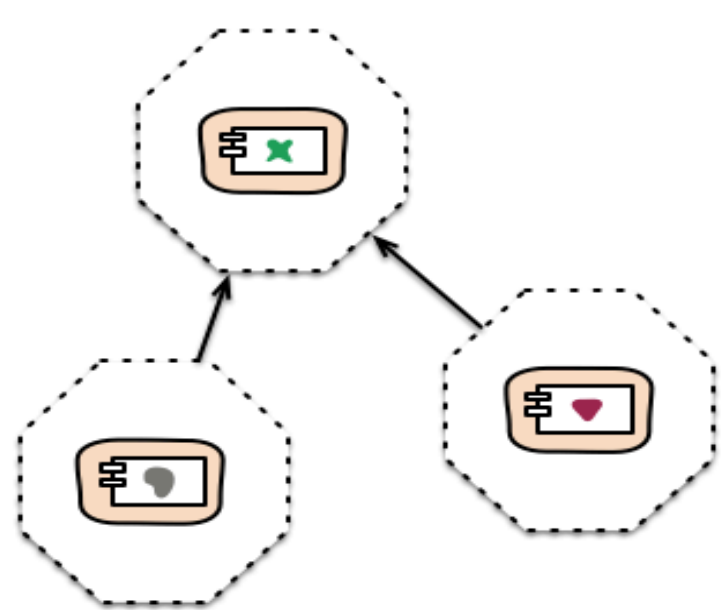
Siloed functional teams...

... lead to siloed application architectures.
Because Conway's Law

Microservices: Avoiding Organizational Coupling

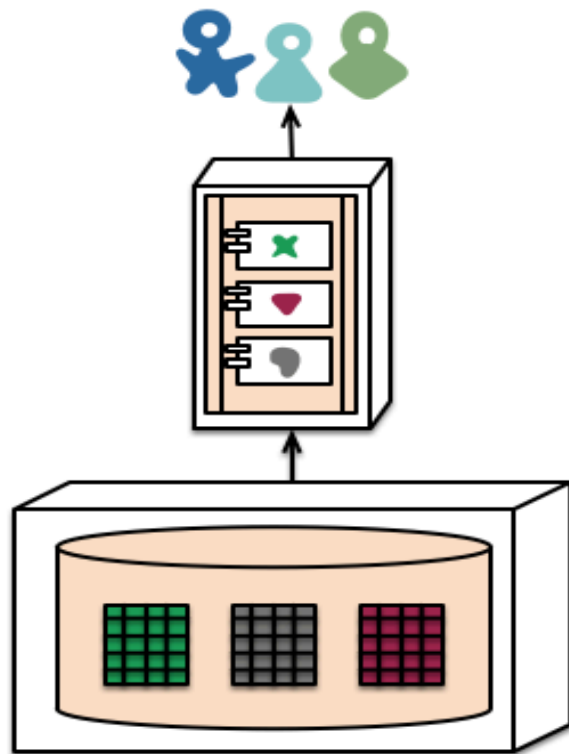


Cross-functional teams...

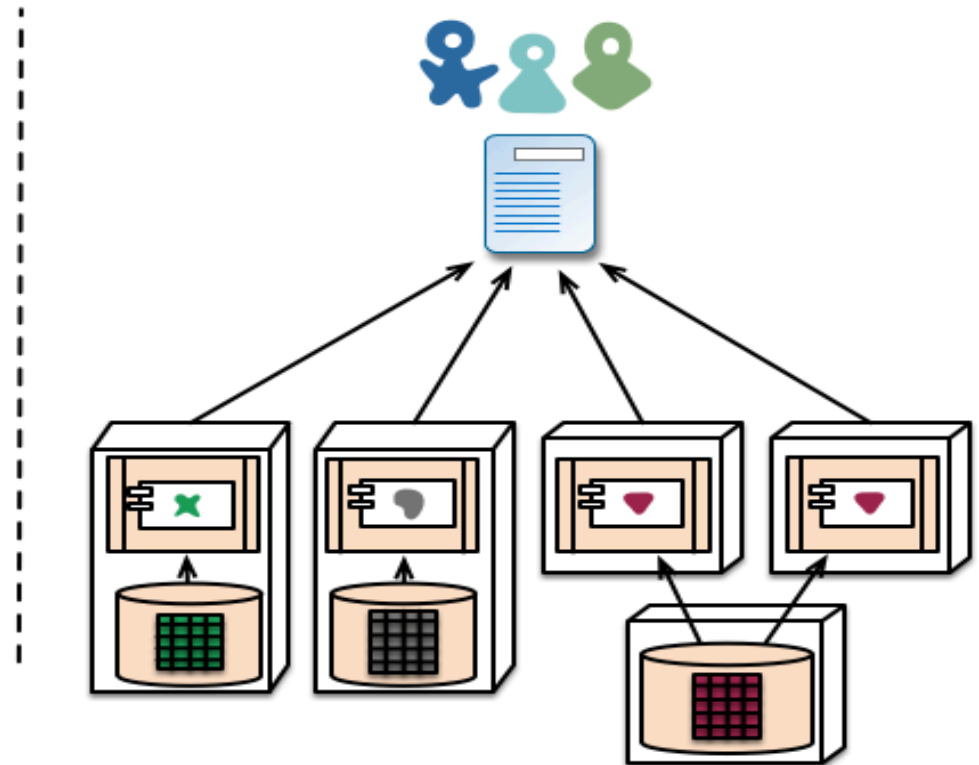


**... organised around capabilities
Because Conway's Law**

Microservices: Data Coupling



monolith - single database



microservices - application databases

Microservices: Properties

- **Polyglot**
 - Multiple languages, platforms, etc.
- **Immutable** code pattern
 - Keep old code / existing service
 - Deploy new version
 - Eventually, re-route traffic
- A/B tests, feature flags, version routing
 - Canary testing
 - Try out new features with developers and testers only
 - Add a few real users, measure improvements
 - Make default for all users, but keep old version

Microservices: Benefits

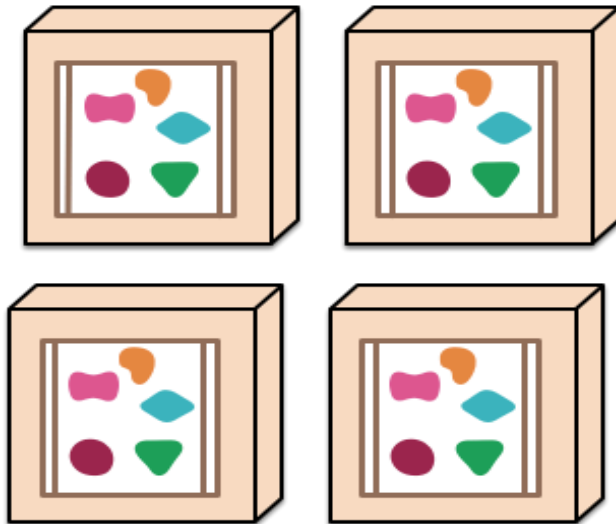
- Small, easy to **understand** code base
- Easy to **scale** (next slide)
- Easy to **throw** away
- Easy to **deploy**
- Easy to use different tools (**polyglot**)
- **Resilient** systems

Microservices: Scaling

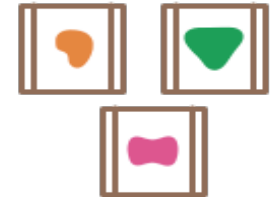
A monolithic application puts all its functionality into a single process...



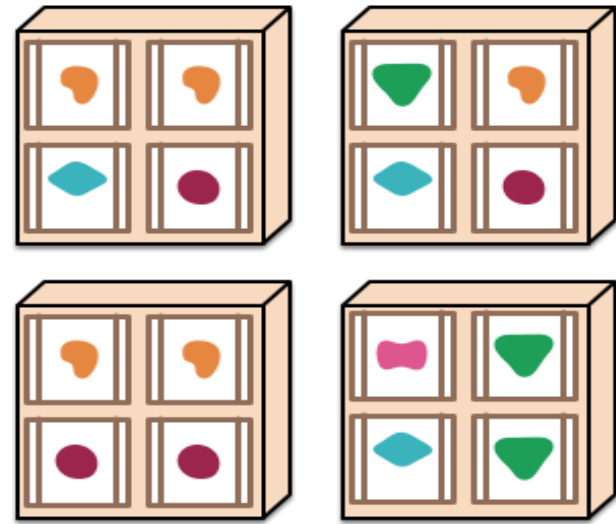
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



Microservices: Drawbacks

- Higher communication **overhead**
 - **Nanoservices** anti-pattern
- **Integration** testing more complicated
- **Moving** responsibility more difficult
 - Needs communication between different teams

Conclusions

- Need to deploy code **rapidly** and **reliably**
- DevOps **bridges** the gap between development and operations
- **Automation** key to success
 - Many tools/services out there to help
- Microservices: preferred architecture for DevOps
- Hot **research** area!
 - Make better use of **operations feedback** during development